

Modelling, Simulation and Virtual validation of Adaptive Cruise Control (ACC) Algorithm based on Sensor Fusion.

Link: <https://eudl.eu/doi/10.4108/eai.17-11-2023.2342869>

Abstract:

Modern Adaptive Cruise Control (ACC) is moving beyond single-sensor limitations by utilising sensor fusion to better navigate complex driving environments.

By merging data from multiple sources, the system identifies the Most Important Object (MIO)—typically the lead vehicle in the same lane. A control algorithm then processes the dynamics of both the "Ego" vehicle and the Lead vehicle to calculate an optimal speed. This target speed is maintained by a longitudinal controller to ensure a safe following distance. To ensure reliability, these systems are validated using virtual closed-loop simulations with real-world driving scenarios.

Key System Components:

- **Sensor Fusion:** Combines data (Radar and Camera) for better object detection.
- **MIO Selection:** Identifies the primary lead vehicle to follow.
- **Stateflow Control:** Calculates the required velocity based on inter-vehicle distance and speed.
- **Longitudinal Controller:** Executes throttle and brake commands to hit the target speed.

Challenges:

The first challenge: When starting to work on the project, where to train and test the algorithm, MATLAB's Roadrunner was typically used, and it is easy to incorporate into the model. **Action:** The ACC was ADAS level 1, which doesn't require complex driving scenarios, so I decided to use the Driving scenario (a built-in app), where the entire scenario was designed to test the system.

The second and main challenge was the lane detection. From the camera data, the lanes are made to binary score images (black and white), but when there is a yellow car in front, the binary conversion has a lot of noise due to the yellow-coloured car in front. **Action:** To resolve this, data from Object detection was used to identify the colour of the car and to nullify the region (binary 0: black), which significantly reduced noise.

The third: Controlling the ego vehicle's speed. The speed of the ego vehicle has to be determined based on multiple inputs from the camera and the radar. Like, what is the target speed, and what distance is a safe distance? **Action:** A stateflow was built with all inputs monitoring the speed and distance in between, and performs corresponding set actions.